Industry Intelligence Skill — System Design

Author: Hephae Engineering **Date**: 2026-03-14 **Status**: Architecture Design — Ready for Review
Depends On: `qualification-pipeline-design.md` (v4)

The existing qualification pipeline and deep discovery system are assumed as-is. This document describes only the industry-specific intelligence layer that sits on top of them.

---

1. Problem Statement

The current pipeline treats every vertical identically. A barber shop, a bakery, and a restaurant all get the same qualification signals, the same scoring weights, the same agent instructions, and the same outreach pitch angles. This creates three problems.

First, generic qualification is inaccurate. An analytics pixel is a strong commercial intent signal for a restaurant running paid ads but nearly meaningless for a barber shop. Instagram presence is the primary portfolio platform for barbers but a secondary channel for bakeries. The pipeline hardcodes signal weights that should vary by industry.

Second, agents lack vertical context. When the SEO agent analyzes a bakery, it doesn't know that bakeries in this county mostly use Wix, that the dominant pain point is "no online ordering despite DoorDash presence," or that the last 40 bakeries analyzed had a median 4.2-star rating. It analyzes in a vacuum, rediscovering patterns that prior runs already revealed.

Third, knowledge doesn't accumulate. Every workflow run starts from zero. Insights from analyzing 50 bakeries in Essex County are not available when analyzing bakeries in Hudson County. There is no mechanism for the system to get smarter over time.

---

2. Core Design Decisions

One Skill, Not One Skill Per Industry

The Industry Intelligence Skill is a single ADK skill. Individual industries — restaurants, barbers, bakeries — are config packages that parameterize the skill, not separate skills.

This distinction matters because the knowledge tools, calibration logic, config loading, and agent wiring are identical across verticals. They just operate on different data. If each industry were a separate skill, every improvement to the knowledge system would need to be replicated N times. With one skill and N config packages, improvements to the intelligence layer automatically benefit every vertical.

Agents Query Knowledge, Python Doesn't Pre-Render It

The previous iteration of this design had Python orchestration code fetching knowledge from Firestore, pre-rendering it into context strings, and injecting those strings into agent prompts. This is anti-agentic — the agent has no ability to ask follow-up questions, request different context, or decide it doesn't need market data for a particular business.

In this design, knowledge is exposed as ADK FunctionTools that agents call on demand. If an SEO agent needs market benchmarks, it calls the benchmark tool. If it doesn't need them because the business's signals are already clear, it skips the call. The agent reasons about what it needs; the tool provides data when asked.

Deterministic Where Possible, Agentic Where Necessary

HTML parsing, chain detection, pixel detection, scoring arithmetic, and signal extraction all stay as deterministic pure functions wrapped in ADK FunctionTools. These do not need LLM reasoning.

The LLM is involved only where judgment is required: the HVT classifier for ambiguous qualification cases, the capability agents in deep discovery, the evaluation agent, and the calibration agent that interprets accumulated outcome data.

Cold Start Graceful Degradation

When no accumulated knowledge exists for an industry and geography, the knowledge tools return empty results. Agents rely entirely on the static instruction set from the industry config. The system works on day one with zero accumulated data; it just gets better over time as runs accumulate.

---

3. System Components

3.1 The Industry Intelligence Skill

This is a single ADK skill that provides four capabilities to the pipeline.

Config Loader: Given a sector string (e.g., "barbers"), loads the corresponding industry config package. Returns the industry-specific instruction sets, signal weight multipliers, platform lists, agent selection, pitch angles, and any custom hook functions. Called once at workflow startup.

Knowledge Tools: A set of ADK FunctionTools registered on pipeline agents, giving them on-demand access to accumulated intelligence from prior runs. There are four knowledge tools, described in section 4.

Calibration Agent: An ADK LlmAgent that runs on a schedule (not per-workflow). It reads outcome data from the knowledge store, identifies patterns in what signals predicted viability,

and updates the benchmark documents. Described in section 6.

Hook Registration: If an industry config package includes custom signal extractors (hook functions), the skill registers them as additional ADK FunctionTools on the relevant agents. Most verticals won't have hooks. The mechanism exists for verticals with unique signals (e.g., "walk-in only" detection for barbers).

3.2 Industry Config Packages

Each config package is a directory containing a YAML config file and an optional hooks file. The config file is purely declarative — it contains no logic, only data that parameterizes the skill.

A config file defines:

Qualification parameters: Signal weight multipliers that adjust the base scoring for this vertical (e.g., boost Instagram for barbers, boost aggregator escape for restaurants). The list of platforms that indicate tech adoption for this vertical. The list of aggregators where this vertical leaks revenue. The definition of "Innovation Gap" for this vertical — what "has tech but missing marketing" looks like. Vertical-specific fast-qualify rules that short-circuit scoring for obvious cases. Additional chain names beyond the shared national list. Relaxed or tightened no-website qualification thresholds. A list of operational friction keywords for review snippet mining (described in Tool 5 under section 4).

Analysis parameters: Which capability agents to run and which to skip for this vertical. Industry-specific instruction text for each active agent — the static baseline knowledge that doesn't depend on accumulated data. For barbers, this tells the SEO agent to focus on local pack ranking and Google Business Profile, tells the social agent that Instagram is the portfolio platform, tells the booking agent to quantify no-show costs.

Outreach parameters: The primary pain points for this vertical, ranked by typical severity. Pitch angle definitions — each with a headline and a list of evidence signals that must be present for the angle to apply. These give the report generation agent vertical-specific framing for the outreach deliverable.

Research parameters: Which demographic and economic indicators from Tier 0 research matter most for this vertical. What "saturation" means in concrete numbers (8 barber shops per zipcode is saturated; 8 restaurants is moderate). These calibrate the dynamic threshold calculation.

The optional hooks file contains one or more pure functions that detect signals unique to this vertical. Each function has a clear name, typed inputs and outputs, and a docstring that ADK can use to describe the tool to agents. The skill registers these as additional FunctionTools when loading the config package.

3.3 Knowledge Store

A Firestore collection structure organized by industry and geography. It holds four types of accumulated data:

Market benchmarks: Aggregate statistics for an industry in a county — dominant platforms, median star rating, average qualification score, viability rate, common pain points, common capability gaps, sample size, which zipcodes have been scanned. Updated incrementally after each workflow run.

Signal effectiveness: Per-signal viability rates — how often each qualification signal (e.g., "has Instagram," "uses Booksy," "has analytics pixel") appeared on businesses that ultimately passed deep discovery as viable. This is the data that allows signal weights to be calibrated empirically rather than guessed.

Exemplars: A curated set of the top-performing business profiles from prior runs. Each exemplar records the qualification score, platform, which signals were present, star rating, review count, gaps found, and pitch angle used. Each exemplar also stores the demographic context of its source geography — income bracket and population density tier from Tier 0 research — so that agents and future matching logic can assess how transferable an exemplar is to a different market. Agents can query exemplars to find comparable businesses and understand what worked.

Outcomes: The raw per-business outcome log. For every business that completes deep discovery, a record is stored with the qualification score, signals present, viability result, failure reason if applicable, and gaps found. This is the input data for the calibration agent.

---

4. Knowledge Tools — What Agents Can Access

Four ADK FunctionTools that query the knowledge store are registered on pipeline agents by the skill, plus one shared deterministic tool for operational friction detection. Agents call these tools when they determine they need context — the tools do not push context to agents.

Tool 1: Get Market Benchmarks

Takes an industry and county. Returns the aggregate benchmark data for that combination: dominant platforms, median ratings, common pain points, sample size. Returns nothing on cold start.

Who calls it: The qualification scorer calls it to understand the competitive baseline. Capability agents call it to benchmark a specific business against the local market norm. The outreach agent calls it to frame the pitch relative to competitors.

Why it's a tool, not pre-injected context: Not every agent needs benchmarks for every business. A barber shop with obvious signals (Booksy + Instagram + 4.8 stars) doesn't need market comparison — the qualification is clear. A borderline case does. The agent decides.

Tool 2: Find Similar Exemplars

Takes an industry, county, and optional filters (platform, minimum star rating, signals present). Returns 2-3 anonymized business profiles from prior runs that match the filter criteria.

Who calls it: Capability agents call it when analyzing a business that fits a recognizable pattern — "is this similar to businesses we've successfully pitched before?" The outreach agent calls it to find the most relevant pitch angle based on what worked for comparable businesses.

Why it matters: Exemplars give agents concrete reference points rather than abstract statistics. An agent can reason "this bakery looks like the one in Nutley that responded well to the aggregator-escape pitch" rather than just seeing a viability rate number.

Tool 3: Get Signal Effectiveness

Takes an industry and county. Returns per-signal viability correlation data — for each qualification signal, how many businesses had that signal, and how many of those were viable.

Who calls it: The qualification scorer calls it to calibrate signal weights based on empirical data rather than static config. The HVT classifier agent calls it when reasoning about ambiguous cases — "is Instagram presence actually predictive for bakeries in this area, or is it noise?"

Why it compounds: After 10 runs, this data is rough. After 50, it's reliable. After 200, it's a precision instrument. The same tool call gets more valuable over time without any code changes.

Tool 4: Record Outcome

Takes all the details of a completed business analysis — qualification score, signals present, viability result, gaps found, pitch angle used. Writes to the outcomes sub-collection in the knowledge store.

Who calls it: The evaluation agent calls it after determining viability. This is how the knowledge store grows — incrementally, per-business, as the pipeline runs. Not in a batch step after the workflow ends.

Why per-business, not per-workflow: If a workflow crashes halfway through, the outcomes for completed businesses are already recorded. No data is lost. It also means the knowledge store is always as current as the last completed business, not the last completed workflow.

Tool 5: Detect Operational Friction (Shared Deterministic)

In addition to the four knowledge tools, the skill provides a shared deterministic tool that scans available text (Google Search snippets, review excerpts surfaced in Tier 1, and website content from Tier 2 probes) for operational friction keywords defined in the industry config.

Each vertical declares its own friction keyword list. For barbers: "long wait," "couldn't get appointment," "no online booking," "had to call," "wait time." For restaurants: "couldn't get a

reservation," "phone was busy," "no one answered," "long hold time." For bakeries: "sold out," "wish I could pre-order," "gone by the time I got there."

The tool is deterministic — regex/substring matching against a keyword list, no LLM needed. It returns which friction phrases were detected and in which source (snippet, review excerpt, website text).

Why this matters for qualification: Friction keywords are demand signals that directly validate pitch angles. A barber shop with no website but 4.8 stars and 15 reviews mentioning wait times is not just a "Get a Website" lead — it's a "Your customers are telling you they want to book online" lead. The friction evidence turns a generic outreach into a pitch backed by the business's own customer feedback.

How it changes disposition: If a no-website business has high ratings and detected friction keywords that match one of the vertical's pitch angles, it qualifies as QUALIFIED_ACTION_REQUIRED with a specialized pitch angle rather than being PARKED. The friction keywords become evidence attached to the pitch angle definition — the outreach agent can reference them directly. For example: "You have a 4.9 rating, but 12 of your recent reviews mention wait times — here's how automated booking would solve that."

Why it's a shared tool, not a per-vertical hook: The friction detection logic is identical across verticals — scan text for keywords from a list. Only the keyword list changes per vertical. Making it a shared tool parameterized by the config avoids duplicating the same scanning logic in every vertical's hooks file. Vertical-specific hooks should be reserved for signals that require genuinely different extraction logic, not just different keyword lists.

---

5. How the Skill Integrates with the Pipeline

5.1 Workflow Startup

When a workflow is triggered for a specific sector and geography, the pipeline asks the Industry Intelligence Skill to load the corresponding config package. The skill returns:

- The instruction sets to inject into each capability agent's system prompt.
- The signal weight multipliers to apply during qualification scoring.
- The list of capability agents to activate and which to skip.
- The set of ADK FunctionTools to register on agents (the four knowledge tools, the friction detection tool parameterized with this vertical's keywords, plus any vertical-specific hooks).
- The pitch angle definitions for outreach.

The pipeline then builds the agent tree for this workflow using these parameters. The pipeline code itself is unchanged — it always builds agents the same way, it just parameterizes them differently based on what the skill returns.

5.2 During Qualification

The qualification scorer uses the skill's signal weight multipliers to adjust base scores. If the scorer has access to signal effectiveness data from the knowledge store (via Tool 3), it blends the static config weights with observed effectiveness. The blending follows a confidence curve:

- Below a minimum sample size: use static config weights only (cold start).
- Above minimum but below a confidence threshold: blend config and observed equally.
- Above confidence threshold: lean heavily on observed data, use config as a prior.

The specific sample size thresholds for each tier of this curve should be tunable per-deployment.

For ambiguous cases that reach the HVT classifier agent, the agent has access to the knowledge tools and can query benchmarks or exemplars to inform its judgment. This is where the agentic design matters most — the classifier can ask "have businesses like this one been viable before?" rather than making a decision in isolation.

5.3 During Deep Discovery

Each capability agent receives two things from the skill: a static instruction set (from the config YAML) and access to the knowledge tools. The instruction set provides baseline vertical knowledge that is always relevant — "for barbers, Instagram is the portfolio platform." The knowledge tools provide accumulated data the agent can query if it decides it needs market context.

The agent decides how to use both. For a straightforward business with clear signals, it may rely entirely on the static instructions and its own analysis. For an unusual or borderline case, it may query exemplars to find comparable businesses, or check benchmarks to understand whether a gap it found is common in this market or an outlier.

The capability agents that run are determined by the skill's config. Barbers get SEO, social, booking flow, and contact. They skip menu analysis and margin surgery. Restaurants get SEO, social, menu analysis, margin surgery, aggregator cost analysis, and contact. Bakeries get a subset similar to restaurants but skip margin surgery. The pipeline's agent tree is assembled at workflow startup based on these declarations.

5.4 During Outreach

The report generation agent uses the skill's pitch angle definitions to select the most relevant framing. Each pitch angle declares what evidence signals must be present for it to apply. The agent matches the business's discovered gaps against the available angles and selects the best fit.

If exemplars are available from the knowledge store, the agent can reference them to strengthen the pitch — "businesses like yours in this area that addressed this gap saw measurable results." The exemplar data provides concrete grounding rather than generic claims.

5.5 After Each Business Completes

The evaluation agent calls the Record Outcome tool with the full result profile. This single write is the entire mechanism by which the knowledge store grows. There is no separate batch aggregation step in the main pipeline — the outcome is written as it happens.

---

6. The Calibration Agent

6.1 Purpose

The calibration agent is an ADK LlmAgent that runs on a schedule — not inside a workflow, but as a separate periodic process (triggered by a Cloud Scheduler cron or after every N completed workflows). Its job is to read raw outcome data and produce updated benchmarks and signal effectiveness statistics.

6.2 Why an Agent, Not a Batch Script

A deterministic batch script could compute the basic aggregates — platform distribution, average scores, signal correlation rates. But the calibration agent can do something a script cannot: reason qualitatively about patterns.

For example, if bakeries with Wix sites in Hudson County consistently fail viability, a script sees a low correlation number. An agent can reason about why — "these bakeries all use the same Wix template with no customization, indicating low digital investment despite having a website. The platform signal is misleading for this market; content depth is the real differentiator." That qualitative insight, written as a note in the benchmark document, is valuable context for future capability agents.

The calibration agent also flags discrepancies between the static config and observed data. If the config says Instagram should be weighted at 1.8 for barbers but observed viability correlation is only 0.52 across 60 businesses, the agent flags this for human review. It does not auto-modify the config — the config is editorial control that humans own.

6.3 What It Reads

The raw outcomes sub-collection for a given industry and county. Each outcome document contains the qualification score, signals present, viability result, gaps found, and pitch angle used for one business.

6.4 What It Writes

Updated benchmark documents: revised platform distribution, median ratings, common pain points, viability rate, sample size.

Updated signal effectiveness documents: per-signal viability rates recomputed from all outcomes.

Calibration notes: free-text observations about patterns, anomalies, or config discrepancies that the agent identified. These notes are readable by other agents through the benchmark tool and by humans through the Firestore console or a future dashboard.

Exemplar curation: the agent identifies the top-performing business profiles and adds them to the exemplars sub-collection, pruning older or less relevant exemplars to keep the set focused.

6.5 Frequency

After every 10 completed businesses for a given industry-county pair, or weekly, whichever comes first. This keeps benchmarks fresh without running calibration on every single outcome.

---

7. The Compounding Loop

This is how the system gets smarter over time. The loop has five steps and it repeats with every batch of completed workflows.

Step 1 — Run: The pipeline processes businesses in a zipcode for a given vertical. Agents use the static config instructions plus whatever accumulated knowledge is available.

Step 2 — Record: As each business completes evaluation, the outcome is written to the knowledge store. Qualification score, signals, viability result, gaps, pitch angle — all captured.

Step 3 — Calibrate: The calibration agent reads the accumulated outcomes, recomputes benchmarks and signal effectiveness, writes qualitative notes about patterns, and curates the exemplar set.

Step 4 — Load: The next workflow for this industry (in the same county or an adjacent one) loads the updated benchmarks through the knowledge tools. Agents now have more data to reason with. Qualification weights are better calibrated. Exemplar comparisons are richer.

Step 5 — Improve: Because qualification is more accurate, fewer businesses fail in deep discovery, which means less wasted compute. Because agents have better context, their analysis is more targeted, which means higher-quality reports. Because pitch angles are grounded in what actually worked for similar businesses, outreach is more effective.

Each revolution of this loop compounds. The first run is generic. The fifth run is informed. The twentieth run is precise.

7.1 Cross-Geographic Transfer

Knowledge is stored per county, but the skill can query adjacent or similar geographies when the local sample size is thin. If a workflow runs for barbers in Bergen County for the first time, but Essex County (adjacent, similar demographics) has 50+ barber outcomes, the knowledge tools can return Essex County data as a fallback with a flag indicating it's from a neighboring area, not the exact market.

Each exemplar stores the demographic context of its source geography (income bracket, population density tier). When an agent receives borrowed exemplars from another county, this metadata is visible in the tool result, allowing the agent to reason about relevance. A bakery exemplar from a wealthy suburban zipcode is a better proxy for another wealthy suburb than for a low-income urban area two miles away, regardless of county boundaries.

In v1, the agent handles this relevance judgment itself based on the metadata. The `find_similar_exemplars` tool does not filter by demographics — it returns exemplars with their demographic metadata attached, and the agent decides how much weight to give them. See the TODO section for the planned "Socio-Economic Mirror" enhancement that adds demographic filtering to the tool itself.

The agent then decides how much to trust borrowed knowledge versus relying on the local static config. The agent reasons about this, which is why borrowed knowledge is surfaced as a tool result with metadata, not silently merged into the benchmarks.

---

8. Where MCP Fits (Not Yet, But Planned)

MCP is not needed for the internal pipeline. Agents communicate with the knowledge store through ADK FunctionTools backed by direct Firestore access. Adding an MCP server between agents and their data would introduce a network hop, serialization overhead, and operational complexity for no benefit.

MCP becomes the right choice when external consumers need access to the accumulated intelligence:

Human-facing research interface: A Claude-powered dashboard where you can ask "What do we know about bakeries in Bergen County?" and get an answer grounded in the knowledge store. The knowledge tools would be wrapped in an MCP server so Claude can call them.

Customer-facing insights: When a business owner asks Hephae "How do I compare to other barber shops in my area?", an MCP server exposes the benchmark data in a customer-safe format.

Third-party integrations: If the industry intelligence becomes a standalone product or data source, MCP provides the interop protocol.

The underlying data layer and accumulation loop are the same regardless of whether MCP sits on top. The migration path is: build the Firestore-backed tools now, wrap them in an MCP server later when there's a consumer that needs it.

---

9. Vertical Selection and Sequencing

9.1 Selection Rationale

The three starting verticals are restaurants, barber shops, and bakeries. This combination is chosen for complementary strengths, not just market opportunity.

Restaurants are the most competitive local business SaaS vertical — Toast, Olo, BentoBox, Popmenu, and Owner.com are all fighting for them. However, restaurants are also where Hephae's pipeline is most mature. The Nutley canary run was all restaurants. The Aggregator Escape bonus, Menu Mismatch auto-qualify logic, margin surgery agent, and commodity pricing integrations are all restaurant-specific work that already exists. Discarding this to avoid competition would waste months of validated infrastructure. Instead, restaurants serve as the proving ground where the qualification pipeline and industry skill system get battle-tested against the richest existing dataset.

Barber shops are the stress test. They are fundamentally different from restaurants — booking-centric instead of ordering-centric, portfolio-driven (Instagram) instead of menu-driven, no food supply chain or aggregator commission dynamics. If the skill config system can cleanly parameterize the pipeline for both restaurants and barbers without code changes, the abstraction is validated. Barbers also have no dominant vertical SaaS competitor, making Hephae's value proposition cleaner and the outreach less crowded.

Bakeries are the bridge between the two. They share the food vertical with restaurants (aggregator escape, online ordering gap, commodity pricing from BLS data, Square/Toast platform landscape) but differ in scale, complexity, and competitive saturation. Fewer SaaS competitors target bakeries specifically. The bakery config will be the fastest to build because it inherits much of the restaurant config's structure with adjustments to scoring weights and pitch angles.

9.2 What Each Vertical Validates

Vertical	Primary validation	Skill config challenge
Restaurants	Pipeline maturity, qualification accuracy, knowledge accumulation loop	Aggregator escape logic, menu-specific signals, highest signal count — tests whether the config handles complexity

Vertical	Primary validation	Skill config challenge
Barbers	Skill abstraction flexibility, fundamentally different signal profile	Booking-centric scoring, Instagram as primary platform, walk-in detection hooks — tests whether a non-food vertical works without pipeline changes
Bakeries	Cross-vertical config reuse, food-vertical generalization	Shares restaurant infrastructure but needs different weights and pitch angles — tests whether similar verticals can diverge cleanly through config alone

9.3 Sequencing

Start with restaurants. The pipeline is already tuned for this vertical, and the qualification pipeline design doc uses restaurants as its reference case. Build the skill system with restaurants as the first config package, validate the full loop (qualification with calibrated weights → deep discovery with knowledge tools → outcome recording → calibration), and confirm that the compounding knowledge loop produces measurable improvement across multiple zipcode runs.

Add barbers second. This forces the skill abstraction to prove itself — if the barber config requires any pipeline code changes, the abstraction is wrong. The barber config should be a YAML file with different signal boosts, different platform lists, different agent instructions, and a hooks file for walk-in detection. Nothing else.

Add bakeries third. This should be the fastest vertical to launch because the config borrows heavily from restaurants. The primary differences are: relaxed no-website thresholds (bakeries are more often high-reputation with no website than restaurants), adjusted saturation definitions (fewer bakeries per zipcode before saturation), and different pitch angle emphasis (online ordering gap over aggregator commission savings).

All three config directories should exist as skeletons from day one so the loader and directory structure are validated early. Only restaurants need a fully fleshed-out config at initial launch.

---

10. What Stays Outside This Design

This document does not cover:

- The qualification pipeline architecture (Tier 0, 1, 2, 3 sieve logic, scoring algorithm, Cloud Tasks fan-out, probe data reuse). That is defined in `qualification-pipeline-design.md` and is assumed as the foundation this skill system integrates with.
- The existing deep discovery agents, evaluation agents, or outreach generation. Those exist and continue to function as-is. The skill system injects instructions and provides tools to them; it does not replace them.

- The specific Firestore document schemas, Pydantic models, or API contracts. Those are implementation details for the coding agent to determine based on this architecture.
- The deployment infrastructure (Cloud Run services, Cloud Tasks queues, Cloud Scheduler for calibration). The calibration agent needs a trigger mechanism, but the choice of scheduler is an infrastructure decision.

---

11. Success Criteria

The system is working when:

1. Adding a new vertical requires only writing a config YAML file (and optionally a hooks file). No pipeline code changes, no new agents, no new deployments.
2. After 50+ businesses processed for a given industry and county, the qualification viability rate (percentage of qualified businesses that pass deep discovery) exceeds 80%, compared to the current ~46% baseline from the Nutley canary.
3. Capability agents in deep discovery produce measurably more targeted analysis when knowledge tools return accumulated data versus cold start — validated by evaluation agent quality scores.
4. The calibration agent successfully identifies at least one signal weight miscalibration (static config vs. observed effectiveness) within the first 100 businesses processed for a vertical.
5. Cross-geographic knowledge transfer works: a new zipcode in a county with existing data produces higher qualification accuracy on its first run than a completely cold-start county.

---

12. TODO — Deferred Enhancements

These are enhancements that have been identified as valuable but are explicitly out of scope for v1. They are documented here so the coding agent understands the future direction and avoids design decisions that would make these harder to add later. Each item describes what it is, why it's deferred, and what the coding agent should do now to keep the door open.

12.1 Socio-Economic Mirror Matching for Exemplars

What: When the `find_similar_exemplars` tool does cross-geographic fallback (section 7.1), it should optionally filter by demographic signals — income bracket, population density — not just county adjacency. A bakery in a high-income suburb in Essex County may find more relevant exemplars in a high-income suburb in Bergen County than in a low-income urban area within Essex County.

Why deferred: Cross-geographic transfer only matters when the local county has thin data and you need to borrow from elsewhere. For that borrowed data to be meaningfully filtered by demographics, you need substantial exemplar volume across multiple counties and demographic tiers. At launch, we'll be operating in a few counties in northern New Jersey with limited data. The matching logic would have nothing to work with.

What to do now: Store income bracket and population density tier (from Tier 0 research) on every exemplar when it is recorded via the Record Outcome tool. This is the critical preparatory step — the demographic fields must exist on the data so that the matching logic can be added later without backfilling. Do NOT build filtering logic in the `find_similar_exemplars` tool yet. In v1, the tool returns exemplars with their demographic metadata attached, and the agent reasons about relevance itself. When exemplar volume is sufficient (likely after 200+ outcomes across 3+ counties), add optional `income_bracket` and `density_tier` filter parameters to the tool.

12.2 Calibration Agent Auto-Tuning Recommendations

What: The calibration agent currently flags discrepancies between static config weights and observed signal effectiveness for human review. In a future version, it should be able to propose specific weight adjustments as a pull request or config change suggestion, with supporting evidence and confidence intervals.

Why deferred: Auto-tuning requires enough outcome data to produce statistically reliable recommendations, and requires trust in the calibration agent's judgment that can only be built through human review of its early qualitative notes. Let the calibration agent run for several months with human oversight before giving it the ability to propose config changes.

What to do now: Ensure calibration notes are stored in a structured format that includes the specific config field in question, the current static value, the observed value, the sample size, and the agent's reasoning. This structured format makes it possible to build an auto-suggestion pipeline later without changing the calibration agent's output format.

12.3 Review Keyword Depth via Dedicated Search Query

What: The friction detection tool (section 4.5) currently scans text that's already available — Google Search snippets, review excerpts in Tier 1 results, and website content from Tier 2 probes. A future version could run a dedicated search query like "[business name] reviews" to pull richer review content from Yelp, Google Reviews, and other sources.

Why deferred: Each additional search query adds API cost and latency to the qualification phase, which is designed to be cheap and fast. The value of deeper review mining needs to be validated against the existing snippet data first. If Tier 1 snippets already surface enough friction keywords to meaningfully change disposition, the dedicated query is unnecessary.

What to do now: Track how often the friction detection tool fires and from which source (snippet vs. review excerpt vs. website text). If, after 100+ businesses, friction signals are rarely detected

because snippets don't contain enough review content, that's the trigger to add the dedicated search query. Log the source of each friction detection in the tool's output so this analysis is possible.

12.4 MCP Server for Knowledge Store Access

What: Wrap the knowledge tools in an MCP server so that external consumers — a Claude-powered dashboard, a customer-facing insights product, or third-party integrations — can query accumulated industry intelligence.

Why deferred: In v1, the only consumers of the knowledge store are the internal pipeline agents, which access it through ADK FunctionTools backed by direct Firestore reads. MCP adds a network hop, serialization, and operational complexity that isn't justified until there's an external consumer.

What to do now: Keep the knowledge tools as clean, self-contained functions with well-defined inputs and outputs. Do not embed pipeline-specific logic (like agent state management or workflow context) into the tool functions. If the tools are pure functions of their inputs, wrapping them in an MCP server later is mechanical — just expose the same functions through MCP's tool protocol. If they're entangled with pipeline internals, the MCP migration becomes a rewrite.

12.5 Cross-Vertical Knowledge Transfer

What: Some signals are industry-agnostic — for example, "businesses with Cloudflare SSL and GTM tags have a higher viability rate regardless of vertical." A future version could maintain a cross-vertical signal effectiveness layer that shares these universal insights across all industry configs.

Why deferred: Distinguishing truly universal signals from vertical-specific ones requires data across multiple verticals in the same geographies. Until all three starting verticals have 50+ outcomes each in overlapping counties, there's not enough data to separate universal patterns from vertical coincidence.

What to do now: Nothing specific. The knowledge store's per-industry-per-county structure doesn't prevent cross-vertical queries — the calibration agent could be extended to read across industries when the data is there. No structural changes are needed to keep this door open.

12.6 Exemplar-Based Outreach Personalization

What: When generating outreach reports, reference specific anonymized exemplar outcomes to strengthen the pitch — "businesses like yours in this area that adopted online booking saw a measurable reduction in no-shows." This grounds the pitch in local evidence rather than generic claims.

Why deferred: Requires enough exemplars with tracked outreach outcomes (did the business respond? did they convert?) to make the claims credible. In v1, we don't yet have outreach

response data flowing back into the knowledge store.

What to do now: Include a `pitch_angle_used` field on every exemplar and outcome record. This is already specified in the Record Outcome tool design. When outreach response tracking is added to the pipeline (a separate project), the exemplar data will already have the field needed to correlate pitch angles with conversion outcomes.